In [ ]:
```python
import marimo as mo
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import polars as pl
import shap
import xgboost as xgb
from pathlib import Path
from sklearn.inspection import permutation_importance

from klarna_ds_case_study.training.config import CATEGORICAL_FEATURES
from klarna_ds_case_study.training.pipeline import (
    load_training_data,
    split_data,
    compute_metrics,
)
```

Feature Engineering & Selection Analysis

Goals:

1. Understand which features drive the current model (importance + SHAP)
2. Identify redundant/collinear features
3. Engineer new ratio/interaction features
4. Benchmark feature subsets and engineered features against the baseline

1. Load data and serving model (calibrated baseline)

In [ ]:
```python
RANDOM_STATE = 42

X, y = load_training_data()
X_train, X_test, y_train, y_test = split_data(X, y)
```

In [ ]:
```python
import mlflow.sklearn

PROJECT_ROOT = Path(__file__).resolve().parents[1]
SERVING_MODEL_DIR = PROJECT_ROOT / "data" / "models" / "serving_model"

baseline_model = mlflow.sklearn.load_model(str(SERVING_MODEL_DIR))

# The serving model is a CalibratedClassifierCV wrapping XGBClassifier.
# Extract the underlying XGB estimator for importance/SHAP analysis.
base_xgb = baseline_model.calibrated_classifiers_[0].estimator
```

In [ ]:
```python
baseline_test_probs = baseline_model.predict_proba(X_test)[:, 1]
baseline_train_probs = baseline_model.predict_proba(X_train)[:, 1]
```

```
baseline_test_metrics = compute_metrics(y_test, baseline_test_probs)
baseline_train_metrics = compute_metrics(y_train, baseline_train_probs)

mo.vstack(
    [
        mo.md(f"**Baseline train metrics:** {baseline_train_metrics}"),
        mo.md(f"**Baseline test metrics:** {baseline_test_metrics}"),
    ]
)
```

Baseline train metrics: {'roc_auc': 0.7159, 'pr_auc': 0.1289, 'log_loss': 0.1957, 'brier_score': 0.0497, 'ece': np.float64(0.0033)}

Baseline test metrics: {'roc_auc': 0.693, 'pr_auc': 0.1187, 'log_loss': 0.1987, 'brier_score': 0.05, 'ece': np.float64(0.0006)}

2. Feature importance (gain + SHAP)

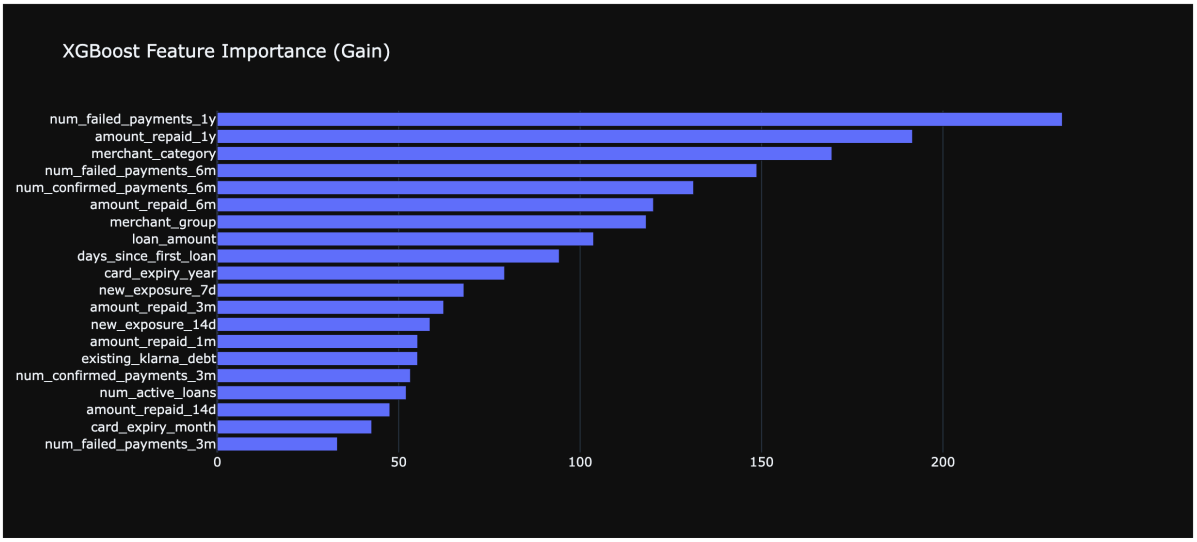
The first thing we look at is xgboost's built-in gain importance, which measures the average gain (i.e. reduction in loss function) of splits using each feature.

It is a useful starting point but can be biased towards features with more splits or higher cardinality. Therefore, we also compute SHAP values, which provide a more consistent feature importance measure by estimating the contribution of each feature to individual predictions.

SHAP values are defined as the average marginal contribution of a feature across all possible subsets of features, giving a more accurate picture of feature influence while accounting for interactions and correlations.

In [ ]:
```
gain_importance = base_xgb.get_booster().get_score(importance_type="gain")
feat_names = X_train.columns.tolist()
gain_df = pd.DataFrame(
    {
        "feature": feat_names,
        "gain": [gain_importance.get(f, 0.0) for f in feat_names],
    }
).sort_values("gain", ascending=False)

fig_gain = go.Figure(
    go.Bar(
        x=gain_df["gain"],
        y=gain_df["feature"],
        orientation="h",
    )
)
fig_gain.update_layout(
    title="XGBoost Feature Importance (Gain)",
    yaxis=dict(autorange="reversed"),
    height=500,
```

```
)
fig_gain
```



Most important features are number of failed and confirmed payments, amount repaid and merchant information.

This aligns with intuition and with the preliminary results from the exploratory analysis.

Least important features, and as such candidates for removal, seem to generally be of the same type as the most important ones, but with shorter time spans.

This suggests that the model is picking up on the same underlying signals across different time windows, and that the shorter-term features may be redundant given the longer-term ones.

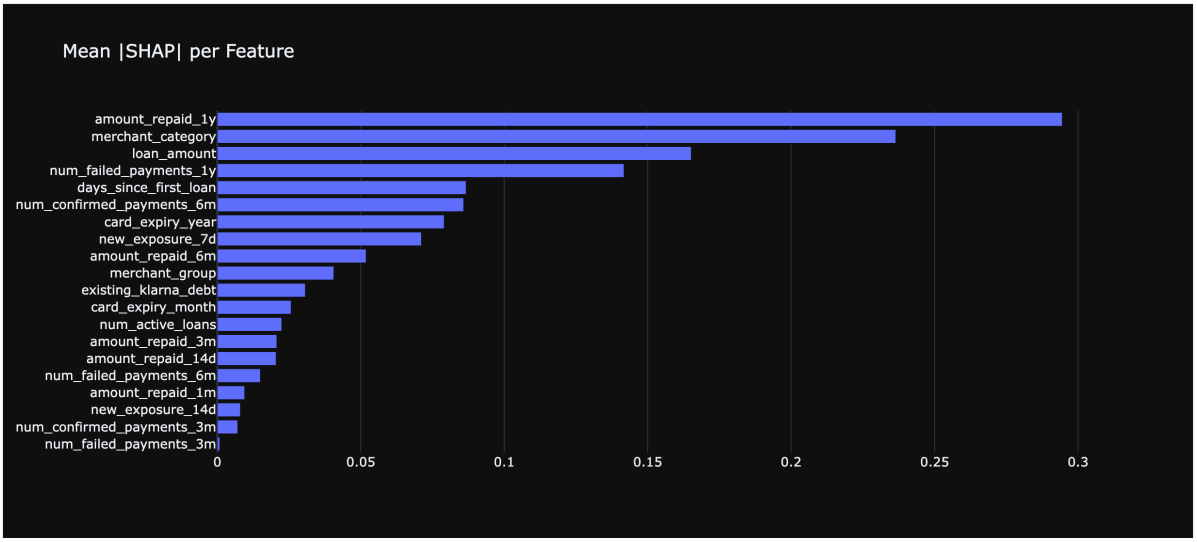
Unexpectedly, card expiry year ranks quite high in the gain importance. It may be a proxy for card age, which could correlate with credit worthiness.

In [ ]:
```
explainer = shap.TreeExplainer(base_xgb)
shap_values = explainer.shap_values(X_train)

mo.md("SHAP values computed on training set.")
```

SHAP values computed on training set.

In [ ]:
```
mean_abs_shap = np.abs(shap_values).mean(axis=0)
shap_df = pd.DataFrame(
    {
        "feature": X_train.columns.tolist(),
        "mean_abs_shap": mean_abs_shap,
    }
).sort_values("mean_abs_shap", ascending=False)

fig_shap = go.Figure(
    go.Bar(
        x=shap_df["mean_abs_shap"],
```

```
        y=shap_df["feature"],
        orientation="h",
    )
)
fig_shap.update_layout(
    title="Mean |SHAP| per Feature",
    yaxis=dict(autorange="reversed"),
    height=500,
)
fig_shap
```

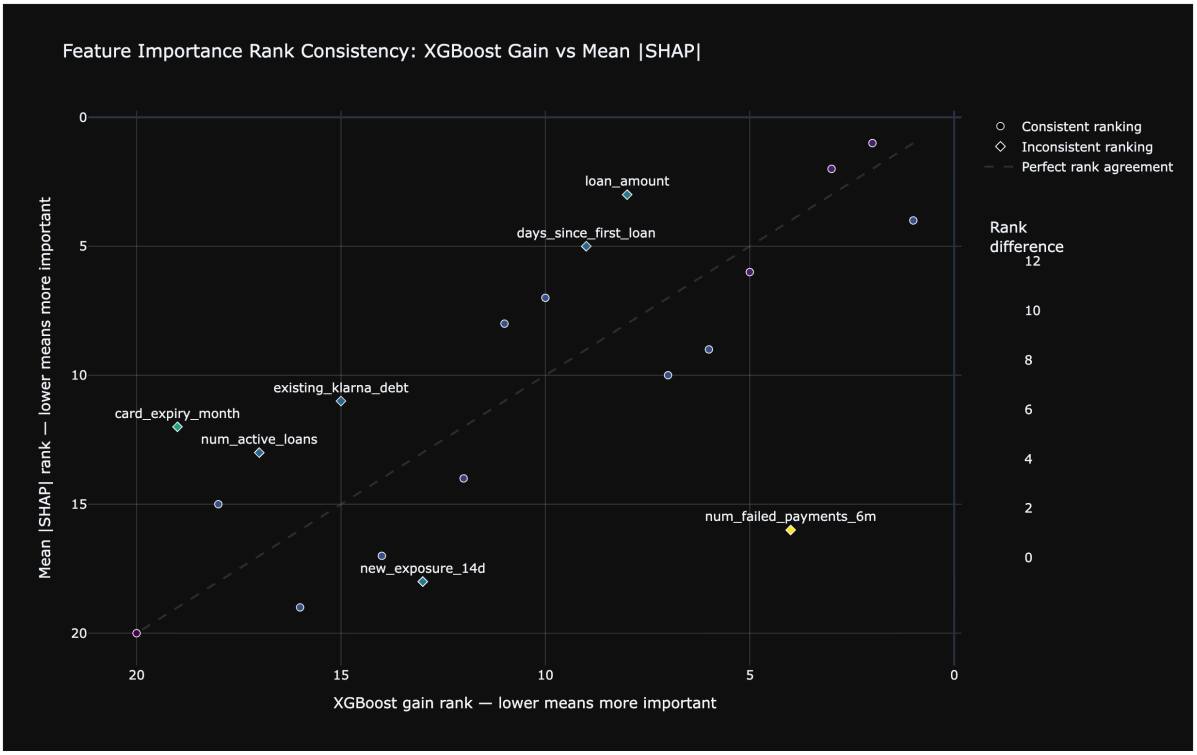


In [ ]:

```
_importance_rank_df = (
    gain_df[["feature", "gain"]]
    .merge(shap_df[["feature", "mean_abs_shap"]], on="feature", how="inner")
    .assign(
        gain_rank=lambda _d: _d["gain"].rank(ascending=False, method="min"),
        shap_rank=lambda _d: _d["mean_abs_shap"].rank(
            ascending=False, method="min"
        ),
        gain_importance_norm=lambda _d: _d["gain"] / _d["gain"].max(),
    )
)

_rank_diff_cutoff = max(3, int(np.ceil(0.20 * len(_importance_rank_df))))
_importance_rank_df["rank_difference"] = (
    _importance_rank_df["gain_rank"] - _importance_rank_df["shap_rank"]
).abs()
_importance_rank_df["ranking_consistency"] = np.where(
    _importance_rank_df["rank_difference"] >= _rank_diff_cutoff,
    "Inconsistent ranking",
    "Consistent ranking",
)
_importance_rank_df["label"] = np.where(
    _importance_rank_df["ranking_consistency"].eq("Inconsistent ranking"),
    _importance_rank_df["feature"],
    "",
)

fig_importance_consistency = px.scatter(
```

```
    _importance_rank_df,
    x="gain_rank",
    y="shap_rank",
    color="rank_difference",
    symbol="ranking_consistency",
    text="label",
    hover_name="feature",
    hover_data={
        "gain": ":.4f",
        "mean_abs_shap": ":.4f",
        "gain_rank": ":.0f",
        "shap_rank": ":.0f",
        "rank_difference": ":.0f",
        "gain_importance_norm": ":.2%",
        "label": False,
    },
    color_continuous_scale="Viridis",
    title="Feature Importance Rank Consistency: XGBoost Gain vs Mean |SHAP|"
    labels={
        "gain_rank": "Gain importance rank",
        "shap_rank": "Mean |SHAP| importance rank",
        "rank_difference": "Absolute rank difference",
        "ranking_consistency": "Ranking consistency",
    },
    height=700,
)

fig_importance_consistency.add_trace(
    go.Scatter(
        x=[1, len(_importance_rank_df)],
        y=[1, len(_importance_rank_df)],
        mode="lines",
        line=dict(color="rgba(80,80,80,0.45)", dash="dash"),
        name="Perfect rank agreement",
        hoverinfo="skip",
    )
)

fig_importance_consistency.update_traces(
    textposition="top center",
    marker=dict(size=7, line=dict(width=0.8, color="white")),
    selector=dict(mode="markers+text"),
)

fig_importance_consistency.update_layout(
    xaxis=dict(autorange="reversed"),
    yaxis=dict(autorange="reversed"),
    legend_title_text="",
    coloraxis_colorbar=dict(title="Rank<br>difference", len=0.65),
)

fig_importance_consistency.update_xaxes(
    title="XGBoost gain rank — lower means more important",
    gridcolor="rgba(180,180,180,0.25)",
)
```

```
fig_importance_consistency.update_yaxes(
    title="Mean |SHAP| rank — lower means more important",
    gridcolor="rgba(180,180,180,0.25)",
)

fig_importance_consistency
```



The two feature importance methods seem to agree overall, with some notable exceptions.

Number of failed payments at 6 months is ranked higher by xgboost, which could possibly be an effect of collinearity with similar features.

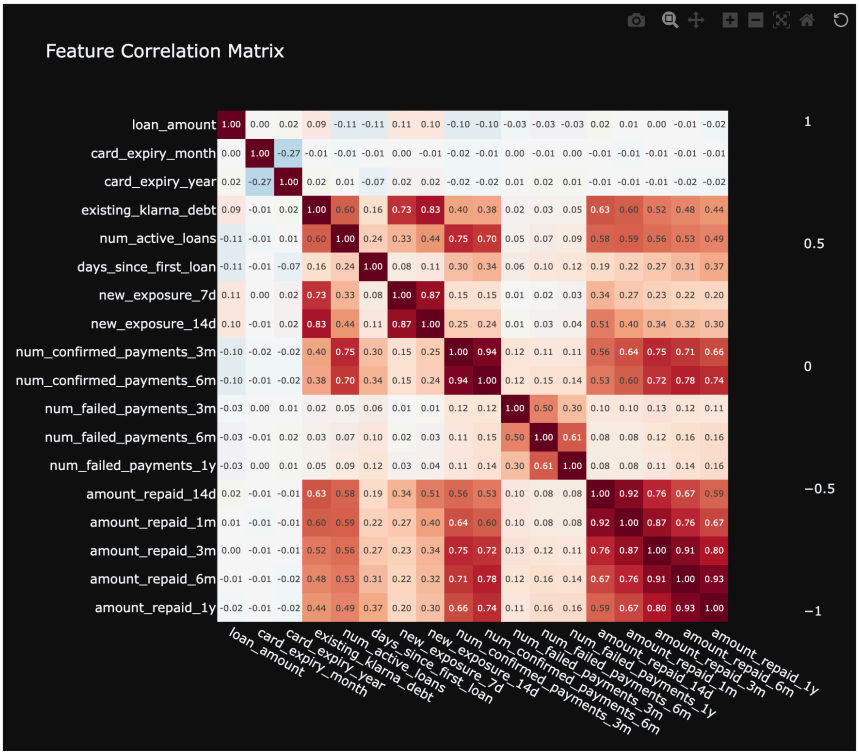
Card expiry month is ranked higher by SHAP, possibly because of interactions with the year feature. Still, it is average.

In conclusion, we collect the least important features on which both methods agree.

In [ ]:
```
least_important_features = [
    "card_expiry_month",
    "amount_repaid_3m",
    "amount_repaid_14d",
    "num_failed_payments_6m",
    "amount_repaid_1m",
    "new_exposure_14d",
    "num_confirmed_payments_3m",
    "num_failed_payments_3m",
]
```

3. Correlation & redundancy analysis

In [ ]:
```
numeric_cols = [c for c in X_train.columns if c not in CATEGORICAL_FEATURES]
corr_matrix = X_train[numeric_cols].corr()

fig_corr = px.imshow(
    corr_matrix,
    text_auto=".2f",
    color_continuous_scale="RdBu_r",
    zmin=-1,
    zmax=1,
    title="Feature Correlation Matrix",
    height=700,
    width=800,
)
fig_corr
```



As expected, many features of the same type are heavily correlated.

New exposure, confirmed payments and amount repaid form the tightest "clusters".

Failed payments is slightly less correlated, and at the same level we also see a cluster with existing debt and number of active loans.

There are also significant correlation among different clusters

In [ ]:
```
# Find highly correlated pairs (|r| > 0.75)
upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astyp
high_corr_pairs = [
```

```
    (col, row, round(upper_tri.loc[row, col], 3))
    for col in upper_tri.columns
    for row in upper_tri.index
    if abs(upper_tri.loc[row, col]) > 0.75
]
high_corr_df = pd.DataFrame(
    high_corr_pairs, columns=["feature_1", "feature_2", "correlation"]
)
high_corr_df = high_corr_df.sort_values("correlation", key=abs, ascending=Fa

mo.md(f"""
**Highly correlated pairs (|r| > 0.75):**

{mo.as_html(high_corr_df)}
""")
```

Highly correlated pairs (|r| > 0.75):

Of the highest-correlated pairs, most of them are of the same type suggesting that dropping one of the two may be the simplest and best solution.

Exceptions are new_exposure_14d vs existing_klarna_debt, amount_repaid_6m vs num_confirmed_payments_6m and num_confirmed_payments_3m vs num_active_loans.

In [ ]:
```
corr_features_to_remove = [
    "num_confirmed_payments_3m",
    "amount_repaid_6m",
    "amount_repaid_1m",
    "amount_repaid_3m",
    "new_exposure_7d",
]
```

4. Feature selection — permutation importance on baseline

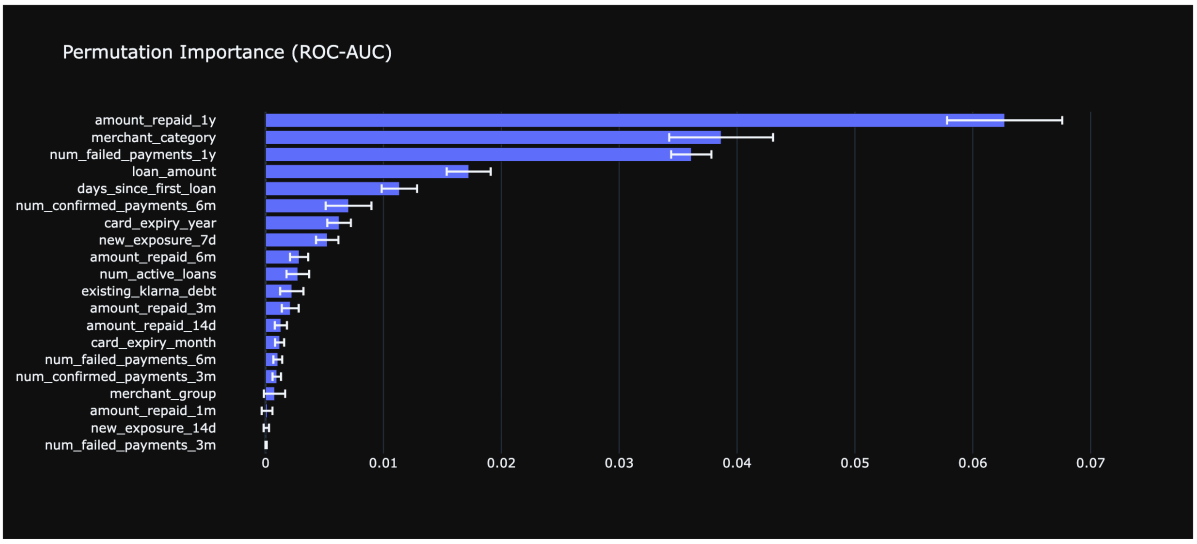
Another way to assess feature importance is permutation importance, which measures the decrease in model performance when a feature's values are randomly shuffled. Features that cause a significant drop in performance are considered important, while those with negligible impact can be candidates for removal.

In [ ]:
```
perm_result = permutation_importance(
    baseline_model,
    X_test,
    y_test,
    n_repeats=10,
    random_state=RANDOM_STATE,
    scoring="roc_auc",
)
perm_df = pd.DataFrame(
    {
```

```
        "feature": X_test.columns.tolist(),
        "importance_mean": perm_result.importances_mean,
        "importance_std": perm_result.importances_std,
    }
).sort_values("importance_mean", ascending=False)

perm_low_importance_features = perm_df[perm_df["importance_mean"] < 0.001][
    "feature"
].tolist()

mo.md(f"""
**Features with near-zero permutation importance (< 0.001 ROC-AUC drop):**

{perm_low_importance_features}
""")
```

Features with near-zero permutation importance (< 0.001 ROC-AUC drop):
['num_confirmed_payments_3m', 'merchant_group', 'amount_repaid_1m', 'new_exposure_14d', 'num_failed_payments_3m']

In [ ]:
```
fig_perm = go.Figure(
    go.Bar(
        x=perm_df["importance_mean"],
        y=perm_df["feature"],
        error_x=dict(type="data", array=perm_df["importance_std"]),
        orientation="h",
    )
)
fig_perm.update_layout(
    title="Permutation Importance (ROC-AUC)",
    yaxis=dict(autorange="reversed"),
    height=500,
)
fig_perm
```



The least important feature are more or less consistent with the previous findings.

A curious exception is merchant_group, which ranks way lower here than it did earlier.

We will keep the list of features to remove as it was already.

In [ ]:
```
least_important_features
```

5. Feature engineering

Creation of engineered features is first and foremost business driven, while still being guided by the insights from the correlation and importance analysis. The goal is to create features that capture meaningful relationships and behaviors, while avoiding redundancy and potential leakage.

In [ ]:
```
def engineer_features(X_in: pd.DataFrame) -> pd.DataFrame:
    Xe = X_in.copy()

    # Ratios
    Xe["debt_to_loan_ratio"] = Xe["existing_klarna_debt"] / Xe["loan_amount"
        lower=1
    )
    Xe["repayment_rate_14d"] = Xe["amount_repaid_14d"] / Xe["loan_amount"].c
        lower=1
    )
    Xe["repayment_rate_3m"] = Xe["amount_repaid_3m"] / Xe["loan_amount"].cli
        lower=1
    )

    # Failed-to-confirmed ratio
    Xe["failed_to_confirmed_3m"] = Xe["num_failed_payments_3m"] / (
        Xe["num_confirmed_payments_3m"] + 1
    )
    Xe["failed_to_confirmed_6m"] = Xe["num_failed_payments_6m"] / (
        Xe["num_confirmed_payments_6m"] + 1
    )

    # Exposure acceleration (second week)
    Xe["exposure_acceleration"] = Xe["new_exposure_14d"] - Xe["new_exposure_

    # Delta features (time-span differences to replace correlated cumulative
    Xe["repaid_delta_1m_minus_14d"] = (
        Xe["amount_repaid_1m"] - Xe["amount_repaid_14d"]
    )
    Xe["repaid_delta_3m_minus_1m"] = Xe["amount_repaid_3m"] - Xe["amount_rep
    Xe["repaid_delta_6m_minus_3m"] = Xe["amount_repaid_6m"] - Xe["amount_rep
    Xe["repaid_delta_1y_minus_6m"] = Xe["amount_repaid_1y"] - Xe["amount_rep
    Xe["failed_delta_6m_minus_3m"] = (
        Xe["num_failed_payments_6m"] - Xe["num_failed_payments_3m"]
    )
    Xe["failed_delta_1y_minus_6m"] = (
        Xe["num_failed_payments_1y"] - Xe["num_failed_payments_6m"]
    )
    Xe["confirmed_delta_6m_minus_3m"] = (
        Xe["num_confirmed_payments_6m"] - Xe["num_confirmed_payments_3m"]
```

```
    )

    # Loan intensity
    Xe["debt_per_active_loan"] = Xe["existing_klarna_debt"] / (
        Xe["num_active_loans"] + 1
    )
    Xe["loan_amount_per_active"] = Xe["loan_amount"] / (Xe["num_active_loans

    return Xe

X_engineered = engineer_features(X)
for _col in CATEGORICAL_FEATURES:
    if _col in X_engineered.columns:
        X_engineered[_col] = X_engineered[_col].astype("category")

new_feature_names = [c for c in X_engineered.columns if c not in X.columns]
print(f"Engineered {len(new_feature_names)} new features: {new_feature_names
```

```
Engineered 15 new features: ['debt_to_loan_ratio', 'repayment_rate_14d', 're
payment_rate_3m', 'failed_to_confirmed_3m', 'failed_to_confirmed_6m', 'expos
ure_acceleration', 'repaid_delta_1m_minus_14d', 'repaid_delta_3m_minus_1m',
'repaid_delta_6m_minus_3m', 'repaid_delta_1y_minus_6m', 'failed_delta_6m_min
us_3m', 'failed_delta_1y_minus_6m', 'confirmed_delta_6m_minus_3m', 'debt_per
_active_loan', 'loan_amount_per_active']
```

6. Benchmark experiments

In [ ]:

```
import json
from itertools import product
from sklearn.calibration import CalibratedClassifierCV

best_params = json.loads(
    (PROJECT_ROOT / "data" / "models" / "best_xgb_params.json").read_text()
)

def evaluate_feature_set(X_all, name):
    Xtr = X_all.loc[X_train.index]
    Xte = X_all.loc[X_test.index]

    spw = float(np.sum(y_train == 0) / np.sum(y_train == 1))
    model = xgb.XGBClassifier(**{**best_params, "scale_pos_weight": spw})
    model.fit(Xtr, y_train)

    calibrated = CalibratedClassifierCV(model, method="isotonic", cv=5)
    calibrated.fit(Xtr, y_train)

    probs_test = calibrated.predict_proba(Xte)[:, 1]
    metrics = compute_metrics(y_test, probs_test)

    return {"experiment": name, "n_features": Xtr.shape[1], **metrics}

original_cols = [c for c in X_engineered.columns if c not in new_feature_nam

results = [
    {
```

```
        "experiment": "baseline (serving model)",
        "n_features": len(original_cols),
        **baseline_test_metrics,
    },
]

for drop_importance, drop_corr, add_engineered in product([False, True], rep
    if not drop_importance and not drop_corr and not add_engineered:
        results.append(
            evaluate_feature_set(
                X_engineered[original_cols], "original (retrained)"
            )
        )
        continue

    parts = []
    cols_to_drop = set()

    if drop_importance:
        cols_to_drop.update(least_important_features)
        parts.append("-importance")
    if drop_corr:
        cols_to_drop.update(corr_features_to_remove)
        parts.append("-corr")

    if add_engineered:
        base_cols = [c for c in X_engineered.columns if c not in cols_to_drc
        parts.append("+engineered")
    else:
        base_cols = [c for c in original_cols if c not in cols_to_drop]

    name = " ".join(parts)
    results.append(evaluate_feature_set(X_engineered[base_cols], name))

results_df = pd.DataFrame(results)
mo.md(f"""
**Benchmark results (test set, all models calibrated with isotonic regressic

{mo.as_html(results_df)}
""")
```

Benchmark results (test set, all models calibrated with isotonic regression):

All feature sets perform similarly, with no boost from engineered features and little to no drop from removing least important and/or highly correlated features.

In conclusion, we decide to keep the minimal feature set due to the lack of evidence of improvement from the engineered features and the benefits of a simpler model with fewer features, such as reduced risk of overfitting, easier interpretability, and lower computational requirements.

In [ ]:
```
final_features_to_remove = list(
    set(least_important_features) | set(corr_features_to_remove)
)
```

In [ ]:
```
final_features_to_remove
```

In [ ]:
```

```